Cuestionario de repaso — Teoría general de UML

Guía de preguntas y respuestas de teoría de UML, independiente de cualquier proyecto o tesis puntual. Pensada como repaso conceptual para un examen.

Índice de temas

1. UML — conceptos generales (11 preguntas)
2. Diagrama de casos de uso (5 preguntas)
3. Diagrama de clases (3 preguntas)
4. Diagrama de secuencia (5 preguntas)
5. Diagrama de actividades (7 preguntas)
6. Diagrama de despliegue (3 preguntas)

---

1. UML — conceptos generales

1.1 ¿Qué es UML?

UML (Unified Modeling Language) es un lenguaje de modelado gráfico, estandarizado por la OMG, para especificar, construir y documentar la estructura y el comportamiento de un sistema de software. No es un método ni una notación de un solo diagrama: es un conjunto de diagramas que representan distintas vistas del mismo sistema, complementarias entre sí.

1.2 ¿Qué diferencia hay entre un diagrama estructural y uno de comportamiento?

Los diagramas estructurales (de clases, de componentes, de despliegue, de objetos) muestran la organización estática del sistema: qué elementos existen y cómo se relacionan entre sí, sin importar el tiempo. Los diagramas de comportamiento (casos de uso, secuencia, actividades, estados, comunicación) muestran cómo interactúan esos elementos y cómo cambia el sistema en el tiempo.

1.3 ¿Qué diferencia hay entre ingeniería directa e ingeniería inversa aplicadas a UML?

La ingeniería directa (forward engineering) parte del modelo UML para generar o guiar la construcción del código — el diseño precede a la implementación. La ingeniería inversa (reverse engineering) parte de un sistema ya construido y produce los diagramas UML que documentan su diseño real — el modelo se deriva del código y del comportamiento observado, no al revés. Ambas son usos legítimos de UML; la elección depende del objetivo (construir algo nuevo vs. documentar/analizar algo existente).

1.4 ¿Qué es la arquitectura de tres capas boundary/control/entity?

Es un patrón clásico de análisis orientado a objetos (propuesto por Ivar Jacobson) para clasificar las clases de un sistema según su responsabilidad: **boundary** (la interfaz con el actor externo, ej. una pantalla o una API), **control** (la lógica de coordinación entre boundary y entity, ej. validaciones y orquestación de un caso de uso) y **entity** (los datos persistentes del dominio del negocio). Cada caso de uso, típicamente, involucra al menos una clase de cada tipo.

1.5 ¿Qué es una clase en UML? ¿Qué son sus atributos y sus métodos?

Una clase es la representación de un conjunto de objetos con la misma estructura de datos y el mismo comportamiento. Se dibuja como un rectángulo dividido en tres compartimentos: el nombre de la clase, sus **atributos** (las propiedades/datos que caracterizan a cada instancia, ej. `nombre: texto`) y sus **métodos/operaciones** (las acciones que la clase puede ejecutar, ej. `guardar()`, `validar()`). Los atributos representan el estado; los métodos representan el comportamiento.

1.6 ¿Qué es un actor en UML?

Un actor es cualquier entidad externa al sistema (una persona, un rol, otro sistema o un dispositivo) que interactúa con él iniciando o participando de un caso de uso. Un actor no es necesariamente una persona física — puede ser un rol que distintas personas cumplen, o incluso un sistema externo o un proceso automático (ej. un cron job que también puede modelarse como actor "Sistema" en un diagrama de actividades).

1.7 ¿Qué es una piscina o carril (swimlane) en un diagrama de actividades?

Es una división visual (vertical u horizontal) de un diagrama de actividades que agrupa las acciones según quién las ejecuta. La regla general de buena práctica es que una piscina debe representar un actor con **agencia real** — alguien o algo capaz de ejecutar la acción por sí mismo (una persona, un rol, un sistema con procesos automáticos) — nunca un lugar físico, un dato o una entidad pasiva que solo se consulta o se referencia.

1.8 ¿Composición y agregación son lo mismo?

No. Ambas son formas de asociación "todo–parte", pero se diferencian por la dependencia de vida entre las partes: en la **agregación** (rombo hueco en el extremo del "todo") la parte puede existir de forma independiente del todo — es una relación de uso o pertenencia débil. En la **composición** (rombo relleno) la parte no tiene sentido ni puede existir sin el todo — si el todo se destruye, sus partes se destruyen en cascada. El ejemplo clásico: una factura (todo) y sus líneas de detalle (partes) son composición, porque una línea de detalle no existe sin su factura.

1.9 Además de asociación, agregación y composición, ¿qué son dependencia y generalización?

La **dependencia** es la relación más débil entre clases: una clase usa a otra de forma puntual (por ejemplo, como parámetro o variable local de un método), sin mantener una referencia permanente — se dibuja con una flecha punteada. La **generalización** es la relación de herencia (clase padre / clase hija, "es un") — se dibuja con una línea llena terminada en un triángulo hueco que apunta hacia la clase padre. A diferencia de la composición/agregación, la generalización no modela "tener partes" sino "ser un tipo de".

1.10 ¿Qué es la transición en un diagrama de actividades?

Es la flecha que conecta dos nodos (estados/acciones, decisiones, barras de sincronización) e indica el paso de uno al siguiente cuando se completa la actividad de origen. Cada actividad, salvo el estado final, debe tener al menos una transición de salida; cada actividad, salvo el estado inicial, debe tener al menos una de entrada. Solo una decisión o una barra de sincronización pueden tener más de una transición de entrada o de salida al mismo tiempo.

1.11 En una herramienta CASE (como StarUML, Enterprise Architect, Visual Paradigm), ¿qué diferencia hay entre el modelo y la vista de un elemento?

Cada elemento (una clase, un caso de uso, un mensaje) existe una única vez como **elemento de modelo** (con su nombre y sus relaciones lógicas dentro del proyecto), y por separado puede tener una o más **vistas** — su representación visual dentro de uno o varios diagramas concretos, enlazadas al modelo por referencia. Esto permite que el mismo elemento de modelo aparezca dibujado en distintos diagramas (por ejemplo, la misma clase en dos diagramas de clase distintos) sin duplicar su definición, y que un elemento exista en el modelo sin tener todavía ninguna vista dibujada ("elemento huérfano").

---

2. Diagrama de casos de uso

2.1 ¿Qué es un caso de uso?

Es una unidad de funcionalidad que el sistema ofrece a un actor y que produce un resultado observable de valor para ese actor (ej. "Registrar Cliente", "Emitir Factura"). Se documenta típicamente con un flujo básico (el camino exitoso), flujos alternativos (variantes y rechazos), y precondiciones/postcondiciones.

2.2 ¿Qué diferencia hay entre `<<include>>` y `<<extend>>`?

`<<include>>` es una relación **obligatoria**: el caso de uso base siempre ejecuta el caso de uso incluido como parte de su flujo (se usa para factorizar comportamiento común, ej. "validar sesión", sin duplicarlo en cada caso de uso). La flecha va del caso base hacia el incluido. `<<extend>>` es una relación **condicional/opcional**: el caso de uso que extiende solo se ejecuta si se cumple una condición puntual en un punto de extensión definido dentro del caso base. La flecha va del caso que extiende hacia el caso base.

2.3 ¿Qué forma tiene siempre un caso de uso en el diagrama?

Un óvalo (elipse), con el nombre del caso de uso adentro o debajo — es la notación fija de UML, independiente de la herramienta que se use para dibujarlo. El actor se dibuja como un muñeco de palitos (stick figure), y un rectángulo puede englobar a todos los casos de uso para representar los límites del sistema.

2.4 ¿Un caso de uso documenta el comportamiento actual de un sistema o el comportamiento deseado?

Depende del propósito del documento. En un proyecto nuevo (ingeniería directa), el caso de uso documenta el comportamiento **deseado**, que todavía no existe, y guía la construcción. En un trabajo de análisis sobre un sistema ya construido (ingeniería inversa), lo habitual es documentar el comportamiento **real verificado** contra el código — salvo que se declare explícitamente que una sección particular describe una mejora o un diseño objetivo distinto al estado actual, lo cual debe quedar señalado sin ambigüedad para no confundir "lo que es" con "lo que debería ser".

2.5 En un caso de uso de "Baja", ¿por qué normalmente no se elimina físicamente el registro, sino que se hace una baja lógica (soft delete)?

Porque un registro maestro casi siempre está referenciado por otros datos históricos del sistema (ventas, consultas, movimientos ya emitidos que citan ese dato). Si se borra físicamente la fila, esas referencias quedan rotas o huérfanas, se pierde trazabilidad/auditoría de lo que existió, y en muchos motores de base de datos la operación directamente falla por restricción de integridad referencial (clave foránea). La **baja lógica** resuelve esto marcando el registro con un campo de estado (activo/inactivo, o una fecha de baja) en vez de eliminar la fila: el dato deja de ofrecerse para nuevas operaciones, pero todo lo que ya lo

referenciaba sigue siendo consistente y consultable. Por esa misma razón, en el diagrama de secuencia el caso "Baja" no dispara un `DELETE`, sino un `UPDATE` sobre el campo de estado — y en el flujo alternativo del caso de uso normalmente no hace falta prever "el registro está en uso" como motivo de rechazo, precisamente porque la baja lógica no rompe nada que lo esté usando.

---

3. Diagrama de clases

3.1 ¿Qué es un diagrama de clases?

El diagrama estructural que muestra las clases del sistema, sus atributos, sus operaciones y las relaciones (asociación, agregación, composición, dependencia, generalización) entre ellas. Es la vista estática por excelencia de UML y suele ser el diagrama de referencia para el resto de los diagramas (los objetos de un diagrama de secuencia son instancias de las clases de un diagrama de clases).

3.2 ¿Cuándo corresponde usar asociación simple, agregación y composición?

Asociación simple: relación estructural entre dos clases sin implicar dependencia de vida ni relación "todo-parte" (ej. un Cliente "realiza" un Pedido). **Agregación** (rombo hueco): relación "todo-parte" donde la parte es independiente y puede existir sin el todo (ej. una Orden referencia un Producto de catálogo, que sigue existiendo aunque se borre la orden). **Composición** (rombo relleno): relación "todo-parte" donde la parte no tiene sentido ni puede existir sin el todo (ej. una Factura y sus líneas de detalle).

3.3 ¿Qué es la multiplicidad en una relación entre clases?

Indica cuántas instancias de una clase pueden asociarse con una instancia de la otra, en cada extremo de la relación. Los valores típicos son `1` (exactamente una), `0..1` (cero o una), `0..` *o simplemente* (cero o muchas) y `1..` *(una o muchas). Por ejemplo, en "un Cliente tiene 0..* Pedidos", el `0..*` del lado de Pedido indica que un cliente puede no tener ningún pedido o tener varios.

---

4. Diagrama de secuencia

4.1 ¿Qué es un diagrama de secuencia y qué muestra que el de casos de uso no muestra?

Muestra el orden temporal exacto de los mensajes que se intercambian los objetos participantes para realizar un caso de uso concreto — algo que el diagrama de casos de uso no detalla, porque este último solo enumera qué hace el sistema, sin describir paso a paso cómo lo hace internamente.

4.2 ¿Qué es una línea de vida (lifeline) y qué es una activación?

La **línea de vida** es el carril vertical que representa la existencia de un objeto/participante a lo largo de la interacción, con su nombre y su clase (`objeto : Clase`) en la parte superior. La **activación** (el rectángulo angosto sobre la línea de vida) representa el período en que ese participante está efectivamente ejecutando código, generalmente en respuesta a un mensaje recibido.

4.3 En un diseño de tres capas boundary/control/entity, ¿qué participantes suele tener un diagrama de secuencia y en qué orden?

Típicamente cuatro líneas de vida en un orden fijo, de izquierda a derecha: el **Actor** (quien inicia la interacción) → la clase **boundary** (la interfaz que recibe la solicitud) → la clase **control** (que valida y

orquesta la lógica) → la clase **entity** (que persiste o consulta los datos). Este orden refleja el flujo real de una solicitud típica: entra por la interfaz, se procesa en el control, y afecta o consulta la entidad.

4.4 Un caso de uso suele tener un solo diagrama de clases pero puede tener varios diagramas de secuencia. ¿Por qué?

El diagrama de clases describe la estructura estática, que no cambia según qué operación se ejecute. El diagrama de secuencia describe un flujo de interacción concreto, y cada operación distinta de un mismo caso de uso (por ejemplo, Alta, Modificación y Baja de una misma entidad) tiene su propia secuencia de mensajes — por eso es normal tener varios diagramas de secuencia (uno por operación) que referencian al mismo diagrama de clases.

4.5 ¿Qué diferencia hay entre un mensaje síncrono y uno asíncrono?

Un mensaje **síncrono** (flecha con punta llena) espera la respuesta antes de continuar — el emisor queda bloqueado hasta recibir el mensaje de retorno (flecha punteada). Un mensaje **asíncrono** (flecha con punta abierta/de línea) se dispara sin esperar respuesta — el emisor sigue su propio flujo en paralelo. Los sistemas web tradicionales de petición/respuesta suelen modelarse con mensajes síncronos; procesos en segundo plano (colas, notificaciones, tareas programadas) suelen modelarse con mensajes asíncronos.

---

5. Diagrama de actividades

5.1 ¿Qué es un diagrama de actividades y en qué se diferencia de uno de secuencia?

Modela el flujo de trabajo (workflow) de un proceso, con foco en decisiones, ramas paralelas y quién ejecuta cada paso — a diferencia del diagrama de secuencia, que modela mensajes entre objetos de software dentro de un caso de uso puntual. El diagrama de actividades tiene una granularidad más cercana al "proceso de negocio" que a la "interacción técnica" entre clases.

5.2 ¿Cuáles son los elementos básicos de un diagrama de actividades?

Estado inicial (un punto de partida único), acción/actividad (un paso del proceso), decisión (un rombo con una condición real, de la que sale más de un camino posible pero solo se toma uno), barra de sincronización o fork/join (una barra que divide un flujo en ramas paralelas o une varias ramas en una sola), flujo final (cierra una rama sin terminar todo el proceso) y estado final (cierra el proceso completo). Las piscinas/carriles (ver 1.7) agrupan estas acciones según quién las ejecuta.

5.3 ¿Cuándo corresponde usar una decisión y cuándo una barra de sincronización (fork/join)?

Una **decisión** se usa cuando hay una bifurcación real: se evalúa una condición y el flujo sigue por **uno solo** de los caminos posibles (mutuamente excluyentes). Una **barra de sincronización** se usa cuando varias ramas ocurren **en paralelo** — como fork, una entrada se abre en varias salidas simultáneas; como join, varias entradas convergen en una sola salida. No deben confundirse: una decisión elige un camino, una barra de sincronización activa/espera varios caminos a la vez.

5.4 ¿Qué diferencia hay entre Flow Final (fin de flujo) y Final State (estado final)?

Flow Final cierra una rama específica del proceso sin terminar el diagrama completo — útil cuando una rama alternativa (por ejemplo, un rechazo de validación) simplemente deja de continuar sin afectar al resto del proceso. **Final State** cierra el proceso completo — debe usarse una sola vez, al final real de

todo el diagrama de actividades.

5.5 ¿Cómo funciona la simetría de la barra de sincronización y qué es la "disparidad de AND"?

La barra de sincronización es simétrica en sus dos usos. Como **fork** (división): una única transición de entrada se abre en dos o más transiciones de salida que corren en paralelo, todas activándose a la vez. Como **join** (unión): dos o más transiciones de entrada convergen en la barra, y de ahí sale una única transición, pero el flujo no continúa hasta que **todas** las ramas de entrada llegaron. Esa exigencia de que se cumplan todas las condiciones de entrada a la vez (no basta con una) es lo que se conoce como lógica AND, en contraste con una decisión, donde solo se sigue un camino (lógica exclusiva/OR).

5.6 ¿Por qué cada diagrama de actividades debe tener un solo estado inicial y un solo estado final?

Porque el diagrama modela un único proceso con un punto de arranque y un punto de cierre bien definidos — tener más de un inicio o más de un final rompe la lectura del proceso como una unidad coherente y genera ambigüedad sobre dónde empieza y dónde termina realmente el flujo completo. Las ramas alternativas que terminan antes se cierran con **Flow Final** (ver 5.4), no con un segundo estado final.

5.7 ¿Qué relación debe existir entre las validaciones del flujo básico y las del flujo alternativo de un caso de uso?

Toda validación que se ejecuta en el flujo básico (por ejemplo, "el sistema valida que el dato no esté duplicado") debe tener su contraparte explícita en el flujo alternativo, describiendo qué ocurre cuando esa validación falla (por ejemplo, "dato duplicado: el sistema rechaza la operación"). El flujo alternativo no es una lista libre de errores: es, en gran medida, el reverso punto por punto de cada validación mencionada en el flujo básico.

---

6. Diagrama de despliegue

6.1 ¿Qué es un diagrama de despliegue y qué muestra que ningún otro diagrama muestra?

Es el único diagrama UML que representa la topología física real de hardware donde corre el sistema: qué nodos (servidores, equipos cliente, dispositivos) existen, qué software se instala en cada uno y cómo se comunican entre sí. A diferencia de los demás diagramas (que son 100% lógicos/de software), el de despliegue es el puente entre el diseño y la infraestructura real de ejecución.

6.2 ¿Qué es un nodo y qué es un artefacto?

Un **nodo** (`Node`) es un elemento de hardware o de entorno de ejecución (un servidor físico o virtual, un equipo cliente, un dispositivo) — se dibuja como un cubo/prisma en 3D. Un **artefacto** (`Artifact`) es la pieza de software concreta que se despliega dentro de un nodo (un ejecutable, un paquete, una base de datos, un archivo de configuración) — se dibuja como un rectángulo con el ícono de documento con la esquina doblada. La relación entre ambos es de contención: el artefacto "vive dentro" del nodo.

6.3 ¿Cómo se conectan los nodos entre sí?

Con una asociación simple, llamada **camino de comunicación** (`communication path`), a veces etiquetada con el protocolo usado (por ejemplo, HTTP, TCP/IP). No implica una flecha de dirección obligatoria como en un diagrama de secuencia: representa el enlace físico o lógico entre dos nodos, no

un flujo de mensajes en el tiempo.